

```
CREATE TABLE dim_date (  
    date_id INTEGER PRIMARY KEY,  
    full_date DATE NOT NULL,  
    year INTEGER,  
    month INTEGER,  
    quarter INTEGER  
);
```

```
CREATE TABLE dim_session (  
    session_id TEXT PRIMARY KEY,  
    date_id INTEGER,  
    chamber TEXT,  
    source_file TEXT,  
    FOREIGN KEY (date_id) REFERENCES dim_date(date_id)  
);
```

```
CREATE TABLE dim_speaker (  
    speaker_id TEXT PRIMARY KEY,  
    speaker_name TEXT,  
    gender TEXT,  
    role TEXT  
);
```

```
CREATE TABLE dim_party (  
    party_id TEXT PRIMARY KEY,  
    party_name TEXT  
);
```

Bridge table because speakers can change party/role over time

```
CREATE TABLE bridge_speaker_party (  
    speaker_id TEXT,  
    party_id TEXT,  
    valid_from DATE,  
    valid_to DATE,  
    PRIMARY KEY (speaker_id, party_id, valid_from),  
    FOREIGN KEY (speaker_id) REFERENCES dim_speaker(speaker_id),  
    FOREIGN KEY (party_id) REFERENCES dim_party(party_id)  
);
```

```
CREATE TABLE fact_utterance (  
    utterance_id TEXT PRIMARY KEY,  
    session_id TEXT,  
    speaker_id TEXT,
```

date_id INTEGER,

text TEXT,

normalized_text TEXT,

word_count INTEGER,

energy_score INTEGER,

security_score INTEGER,

economic_score INTEGER,

climate_score INTEGER,

minerals_score INTEGER,

energy_density FLOAT,

security_density FLOAT,

economic_density FLOAT,

climate_density FLOAT,

minerals_density FLOAT,

primary_framing TEXT,

is_energy_security BOOLEAN,

is_energy_minerals BOOLEAN,

securitization_index FLOAT,

FOREIGN KEY (session_id) REFERENCES dim_session(session_id),

FOREIGN KEY (speaker_id) REFERENCES dim_speaker(speaker_id),

FOREIGN KEY (date_id) REFERENCES dim_date(date_id)

);

EXAMPLE QUERIES

Energy-security framing over time

SELECT

d.year,

AVG(f.security_density) AS avg_security_density,

AVG(f.securitization_index) AS avg_securitization

FROM fact_utterance f

JOIN dim_date d ON f.date_id = d.date_id

GROUP BY d.year

ORDER BY d.year;

Top speakers in energy-security debates

SELECT

s.speaker_name,

COUNT(*) AS utterances,

```
    AVG(f.security_density) AS avg_security_density
FROM fact_utterance f
JOIN dim_speaker s ON f.speaker_id = s.speaker_id
WHERE f.is_energy_security = TRUE
GROUP BY s.speaker_name
ORDER BY utterances DESC;
```

Top speakers in energy-security debates

```
SELECT
    s.speaker_name,
    COUNT(*) AS utterances,
    AVG(f.security_density) AS avg_security_density
FROM fact_utterance f
JOIN dim_speaker s ON f.speaker_id = s.speaker_id
WHERE f.is_energy_security = TRUE
GROUP BY s.speaker_name
ORDER BY utterances DESC;
```

Party-level framing comparison

```
SELECT
    p.party_name,
    AVG(f.security_density) AS security,
    AVG(f.economic_density) AS economic,
    AVG(f.climate_density) AS climate
FROM fact_utterance f
JOIN dim_speaker s ON f.speaker_id = s.speaker_id
JOIN bridge_speaker_party bsp ON s.speaker_id = bsp.speaker_id
JOIN dim_party p ON bsp.party_id = p.party_id
GROUP BY p.party_name;
```

To be more scalable

```
CREATE INDEX idx_fact_date ON fact_utterance(date_id);
CREATE INDEX idx_fact_speaker ON fact_utterance(speaker_id);
CREATE INDEX idx_fact_session ON fact_utterance(session_id);
CREATE INDEX idx_fact_primary_framing ON fact_utterance(primary_framing);
CREATE INDEX idx_fact_energy_security ON fact_utterance(is_energy_security);
CREATE INDEX idx_date_year ON dim_date(year);
```